

COMUNICAÇÃO PERIÓDICA EM VEÍCULOS AUTÔNOMOS

P5 Grupo M2



08 de junho de
2026

OBJETIVO ATUAL: MENSAGENS TIPADAS E PERIÓDICAS

- A comunicação entre componentes e veículos deve ser feita através de mensagens de Interesse e de Resposta em um modelo similar ao publish-subscribe, mas sem publicações explícitas.
- Agentes do sistema enviam mensagens de Interesse (I) em receber dados de um tipo de acordo com um período
- Agentes que produzem um tipo de dado enviam Respostas (R) aos interessados nesse tipo com frequência referente ao período exigido pelos interessados.

REQUISITOS DE IMPLEMENTAÇÃO DO P5:

- Mensagens devem possuir um **código** que identifique inequivocadamente o tipo de dado a que se referem
- **Mensagens:** Nesta etapa, as mensagens de Interesse e Resposta seguem as seguintes estruturas:
 - I = {kind=INTEREST, origin, timestamp, type, **period**}
 - R = {kind=RESPONSE, origin, timestamp, type, **value**}

Comportamento esperado dos agentes

1. Monitorar interesses

Todo agente capaz de produzir dado do tipo T deve escutar Interesses com $\text{type} = T$.

2. Iniciar envio periódico

Ao receber um Interesse, iniciar envio de Respostas no intervalo period especificado.

3. Gerenciar o timer

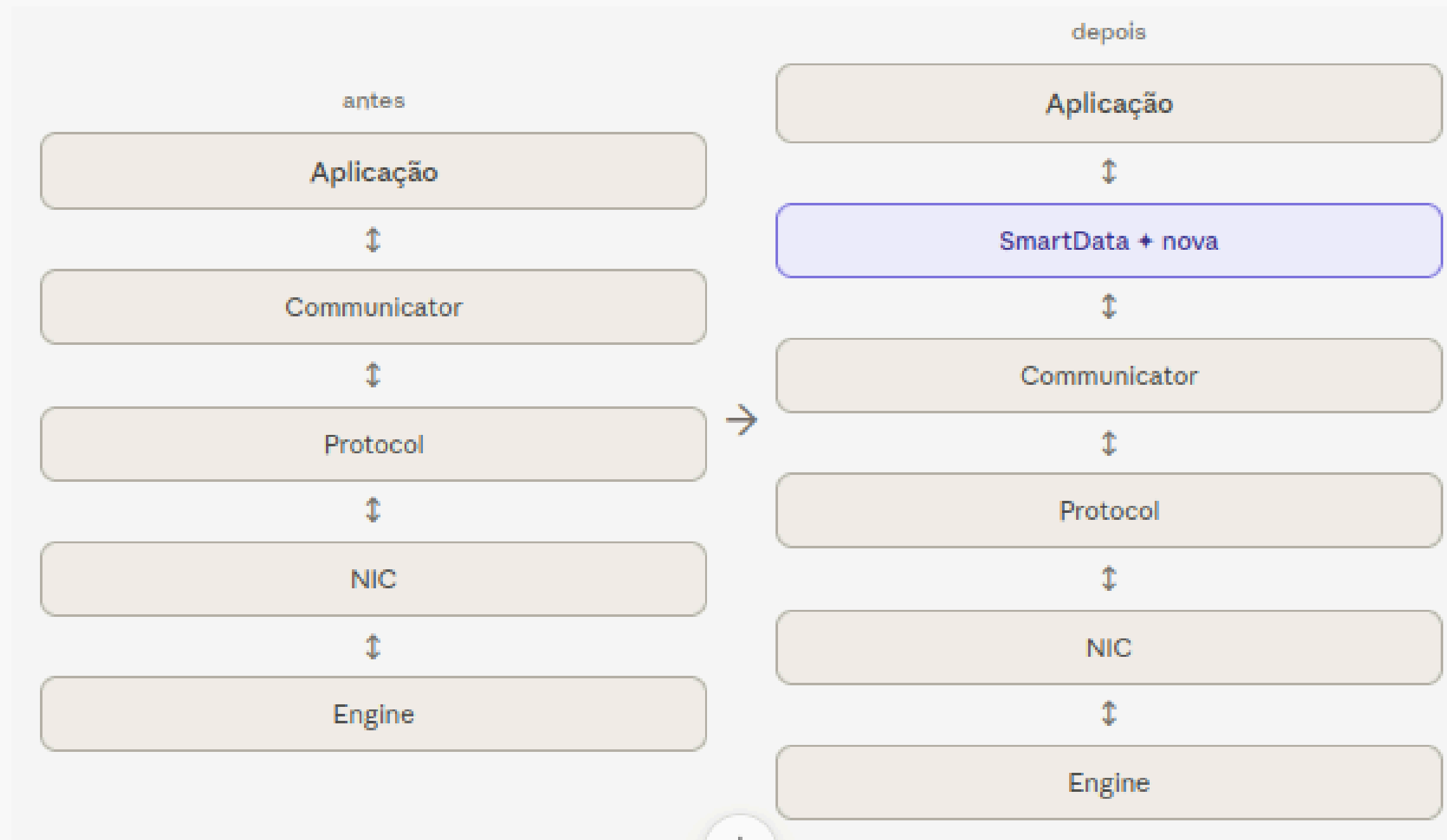
Cada agente pertinente gerencia seu próprio intervalo internamente, independentemente dos demais.

PRINCIPAIS ADIÇÕES:

- Criação de camada **SmartData** entre Communicator e Aplicação
- Criação da enum Unit que identifica tipos de dados inequivocadamente.
- Porta derivada do **UNIT_CODE** para roteamento das mensagens
- **InterestedSmartData<UNIT>** e **ResponsiveSmartData<Transducer>** parametrizados por tipo de dado
- Criação de lógica de declaração de interesse e resposta periódica via **PeriodicThread**
- Criação de **novos componentes** com base nos novos tipos de dados

Nova pilha de comunicação

SmartData passa a ser a camada entre aplicação e Communicator



SMARTDATA - A NOVA CAMADA

Cada componente tem um Interested por dado que consome e um Responsive por dado que produz.

InterestedSmartData — quer receber um tipo de dado

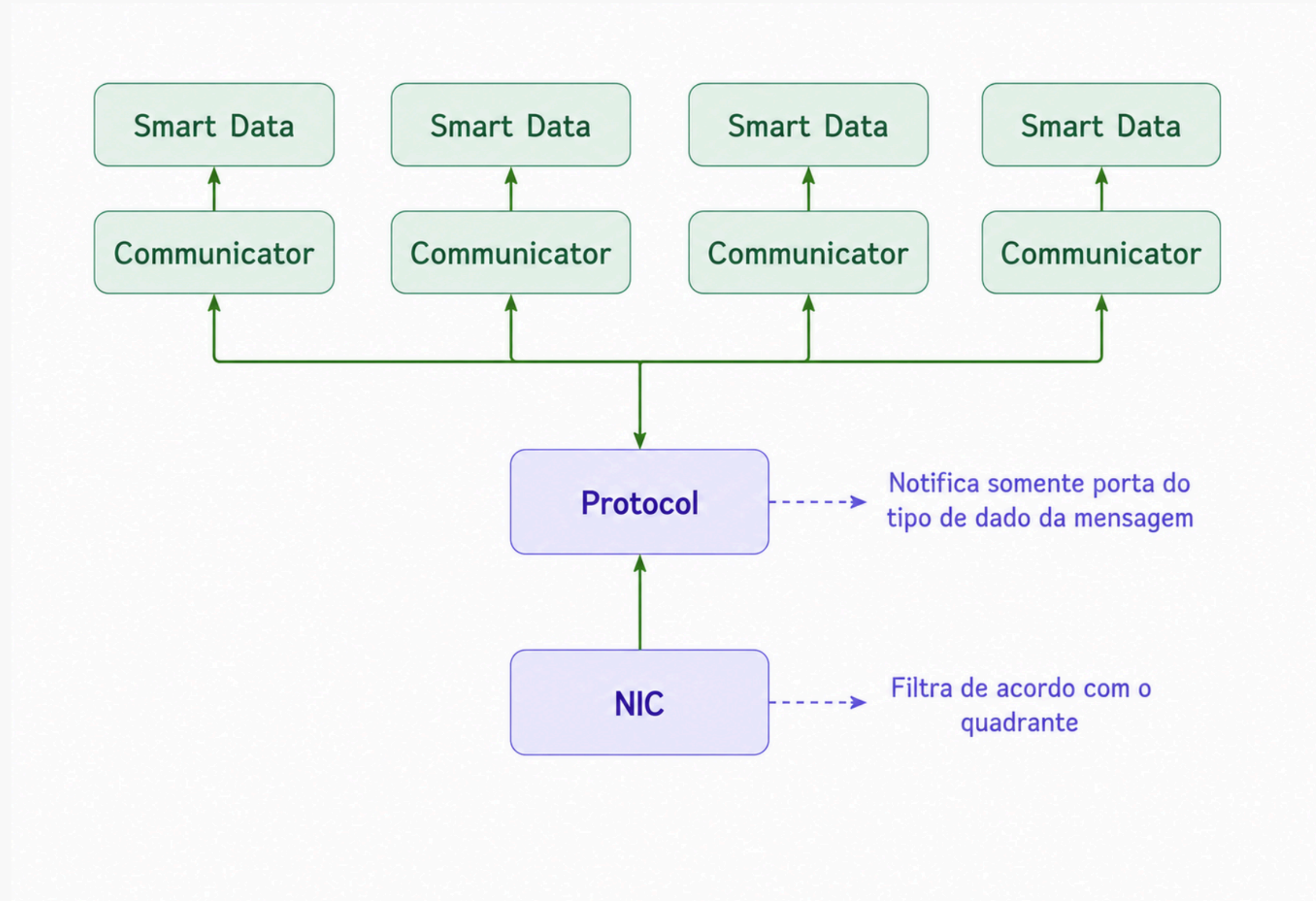
- 1. Envia Interest em broadcast com { kind, type, period }
- 2. Tem Communicator inscrito na porta = UNIT_CODE
- 3. Bloqueia em receive() aguardando Responses
- 4. Ao receber Response: registra timestamp e processa valor

ResponsiveSmartData — produz um tipo de dado

- 1. Tem Communicator inscrito na porta = UNIT_CODE
- 2. Ao receber Interest, extrai o período solicitado
- 3. Se período novo: cria PeriodicThread que envia
- Responde periodicamente

```
Communicator  
porta = UNIT_CODE (ex: TEMPERATURE)
```

Cada Communicator pertence a um InterestSmartData / ResponsiveSmartData



IDENTIFICAÇÃO DE TIPOS : UNIT ENUM

- type é um uint32_t de 32 bits codificado segundo o IEEE 1451 TEDS
- Cada bit tem significado: tipo numérico (I32/I64/F32/D64), modo (DIR/DIV/LOG), e expoentes das unidades SI base (m, kg, s, A, K, mol, cd)
- Exemplos: Temperature = 0xE4924924 (Kelvin), Acceleration = 0xE4924... (m/s²)
- O type identifica inequivocamente o dado — dois **sensores com o mesmo type produzem dados comparáveis e intercambiáveis**
- Usado como porta do Communicator — **roteamento automático por tipo de dado**

```
enum Quantity: Code {
    //          num mod,  sr,   rad,   m,   kg,   s,   A,   K,   mol,   cd      unidade SI correspondente
    Acceleration = SIUnit<D64, DIR,  +0,  +0,  +1,  +0,  -2,  +0,  +0,  +0,  +0>::UNIT, // m/s2
    Angle        = SIUnit<D64, DIR,  +0,  +1,  +0,  +0,  +0,  +0,  +0,  +0,  +0>::UNIT, // rad
    Angular_Velocity = SIUnit<D64, DIR, +0,  +1,  +0,  +0,  -1,  +0,  +0,  +0,  +0>::UNIT, // rad/s
    Area         = SIUnit<D64, DIR,  +0,  +0,  +2,  +0,  +0,  +0,  +0,  +0,  +0>::UNIT, // m2
    Current      = SIUnit<D64, DIR,  +0,  +0,  +0,  +0,  +0,  +1,  +0,  +0,  +0>::UNIT, // Ampere
    Electric_Current = Current,
    Force        = SIUnit<D64, DIR,  +0,  +0,  +1,  +1,  -2,  +0,  +0,  +0,  +0>::UNIT, // Newton
    Humidity     = SIUnit<D64, DIR,  +0,  +0,  -3,  +1,  +0,  +0,  +0,  +0,  +0>::UNIT, // kg/m3
    Length       = SIUnit<D64, DIR,  +0,  +0,  +1,  +0,  +0,  +0,  +0,  +0,  +0>::UNIT, // m
    Luminous_Intensity = SIUnit<D64, DIR, +0,  +0,  +0,  +0,  +0,  +0,  +0,  +0,  +1>::UNIT, // cd
    Mass         = SIUnit<D64, DIR,  +0,  +0,  +0,  +1,  +0,  +0,  +0,  +0,  +0>::UNIT, // kg
    Power        = SIUnit<D64, DIR,  +0,  +0,  +2,  +1,  -3,  +0,  +0,  +0,  +0>::UNIT, // Watt (kg.m2/s3)
    Pressure     = SIUnit<D64, DIR,  +0,  +0,  -1,  +1,  -2,  +0,  +0,  +0,  +0>::UNIT, // Pascal (kg/m.s2)
    Velocity     = SIUnit<D64, DIR,  +0,  +0,  +1,  +0,  -1,  +0,  +0,  +0,  +0>::UNIT, // m/s
    Sound_Intensity = SIUnit<D64, DIR, +0,  +0,  +0,  +1,  -3,  +0,  +0,  +0,  +0>::UNIT, // W/m2 (kg/s3)

    Temperature  = SIUnit<D64, DIR,  +0,  +0,  +0,  +0,  +0,  +0,  +1,  +0,  +0>::UNIT, // Kelvin
    Time         = SIUnit<D64, DIR,  +0,  +0,  +0,  +0,  +1,  +0,  +0,  +0,  +0>::UNIT, // s
    Speed        = Velocity,
    Volume       = SIUnit<D64, DIR,  +0,  +0,  +3,  +0,  +0,  +0,  +0,  +0,  +0>::UNIT, // m3
    Voltage      = SIUnit<D64, DIR,  +0,  +0,  +2,  +1,  -3,  -1,  +0,  +0,  +0>::UNIT, // Volt
    Water_Flow   = SIUnit<D64, DIR,  +0,  +0,  +3,  +0,  -1,  +0,  +0,  +0,  +0>::UNIT, // m3/s
    Frequency    = SIUnit<D64, DIR, +0,  +0,  +0,  +0,  -1,  +0,  +0,  +0,  +0>::UNIT, // Hz

    Ratio        = SIUnit<D64, LOG_DIV, -4,  -4,  -4,  -4,  -4,  -4,  -4,  -4,  -4>::UNIT, // não é SI, um valor adimen
    Percent      = SIUnit<D64, LOG_DIV, -4,  -4,  -4,  -4,  -4,  -4,  -4,  -4,  -3>::UNIT, // não é SI, um valor adimen
};
```


PORTAS BASEADAS EM TIPO • FILTRAGEM NA PROTOCOL

Porta = UNIT_CODE (código IEEE 1451 do tipo de dado)

Fluxo de filtragem:

NIC recebe frame

↓

Filtra por quadrante (src_quadrant)

↓

Protocol extrai dst_port do header

↓

Há Communicator registrado na porta? — SIM → notifica SmartData correspondente
— NÃO → descarta frame

- Temperature → porta 0xE492xxxx
- Velocity → porta 0xE452xxxx
- Pressure → porta 0xE412xxxx

Interest e Response usam a mesma porta (UNIT_CODE).

O campo kind no payload distingue os dois tipos.

LÓGICA DE INTERESSE & RESPOSTA PERIÓDICA

Um período novo → uma thread nova. Threads independentes por período.

InterestedSmartData envia em broadcast:

```
I = {  
  kind      = INTEREST,  
  origin    = meu endereço,  
  timestamp = agora,  
  type      = TEMPERATURE,  
  period    = 500000 // µs  
}
```

bloqueia aguardando Respostas na porta TEMPERATURE

ResponsiveSmartData ao receber I:

1. Extrai period do Interest
2. Já tem thread para esse período?
 - SIM → ignora
 - NÃO → cria PeriodicThread

```
R = {  
  kind      = RESPONSE,  
  origin    = meu endereço,  
  timestamp = agora,  
  type      = TEMPERATURE,  
  value     = 298.15  
}
```

Thread envia a cada period µs

```
===== PTP STATS =====  
PTP sync OK: 64  
PTP timeouts: 1  
Ultimo offset: 548950 ns  
=====
```

```
==== POSITION STATS ====  
First Quadrant: 0  
Last Quadrant: 3  
Q0: 21.5%  
Q1: 12.3%  
Q2: 21.5%  
Q3: 44.6%  
=====
```

```
===== SMARTDATA STATS =====  
Responses recibidos (3835050276): 41  
Last value: 26.1  
Average: 15.3122  
=====
```

```
===== SMARTDATA STATS =====  
Responses recibidos (3835046180): 41  
Last value: 13.5  
Average: 10.4049  
=====
```

```
===== SMARTDATA STATS =====  
Responses recibidos (3834554660): 41  
Last value: 99220  
Average: 100249  
=====
```

```
===== SMARTDATA STATS =====  
Responses recibidos (3834792740): 0  
Last value: N/A  
Average: N/A  
=====
```

OPÇÕES DE TESTE

- **make**: execução com 20 veículos, 5 componentes cada
- **make test_base_local**: 1 interessado, 1 responsivo
- **make test_silent_producer**: sem pares de interessado e responsivo
- **make test_fanout_local**: N interessados, 1 responsivo

OBRIGADO PELA ATENÇÃO!

P5 Grupo M2



08 de junho de
2026